

R Tutorial to fit unimodal spline method from the paper “Flexible unimodal density estimation in hidden Markov models

1. Setup

In this R tutorial, we illustrate the implementation of the unimodal spline method described in the paper “Flexible unimodal density estimation in hidden Markov models”. In that paper, we develop a method to fit hidden Markov models with flexible, P-spline-based emission densities that are constrained to be unimodal. We demonstrate how such constraints can make the model more robust against slight misspecification of the state process model which is often the case in practice.

The corresponding .Rmd file can be found [here](#).

We begin by first loading relevant packages and sourcing custom helper functions.

```
library(splines2)
library(LaMa)
```

2. Understanding the mode location in unimodal splines

We place unimodality restrictions on a vector of coefficients in order to restrict a density function to be unimodal. This section explains the connection between these two restrictions, and how the location of the mode of the function is not easily relatable to the restrictions on the coefficients. This section, which contains some technical details, can be skipped at first reading.

We start by defining a function f in terms of a B-spline expansion, that is

$$f(x) = \sum_{k=1}^K \alpha_k B_k(x),$$

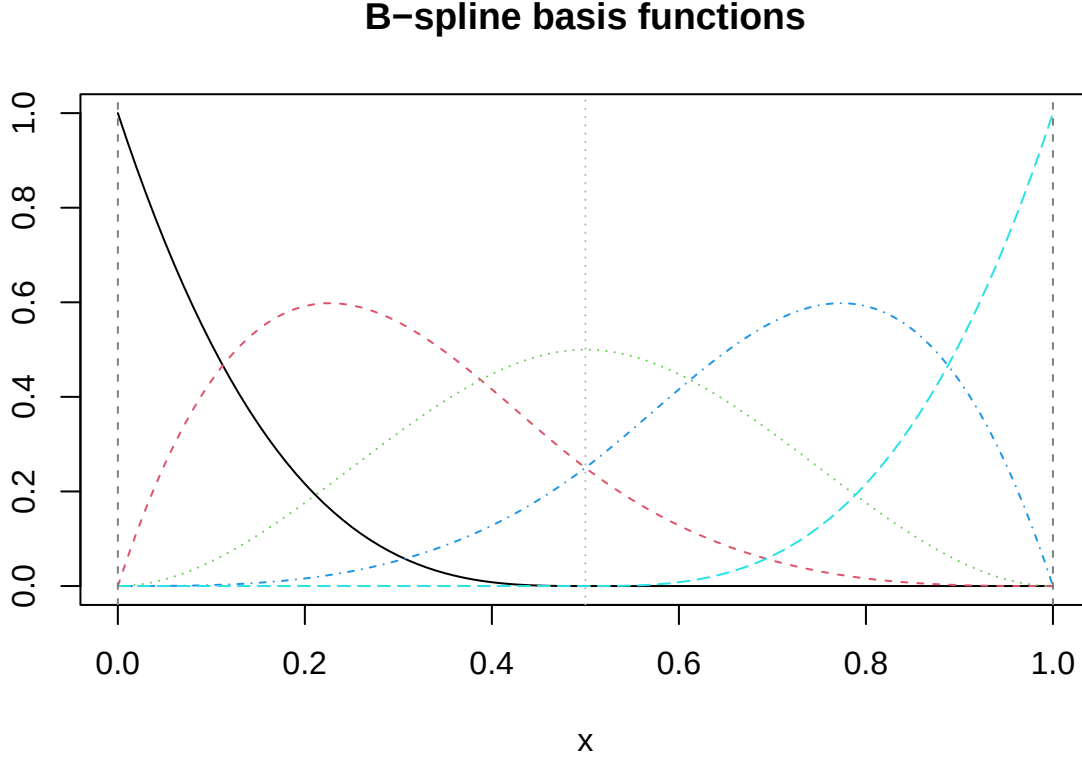
where $B_1, \dots, B_K$ are B-spline basis functions and $\alpha_1, \dots, \alpha_K$ are the spline coefficients, controlling the shape of f . Let us take a moment to think through how the mode location of f interacts with a mode in the coefficient sequence $\alpha_1, \dots, \alpha_K$ of the expansion. We start with a basic example. Suppose we have 5 B-spline basis functions defined on $[0,1]$, which means we also have 5 corresponding spline coefficients, denoted as $\alpha_1, \alpha_2, \alpha_3, \alpha_4, \alpha_5$. These basis functions are typically defined by a set of knots, and for simplicity, let's assume just one interior knot at $x = 0.5$. This knot splits the domain $[0,1]$ into two intervals:

- $[0,0.5)$
- $[0.5,1]$

```
knots <- seq(0, 1, by = 0.5)
knots <- knots[2:(length(knots)-1)] # interior knot sequence
# Note: bSpline sets default boundary knots as max, min of x - but user can specify
degree <- 3 # degree of the B-spline basis - here cubic
x <- seq(0, 1, by = 0.01) # evaluation points
bsMat <- bSpline(x, knots = knots, Boundary.knots = c(0,1), degree = degree, intercept = TRUE)
# bsMat = design matrix: bsMat[i,j] = phi_j(x_i)
# dimension of bsMat = length(x) by degree + 1 + length(knots)
```

Let's take a look at the 5 basis functions.

```
plot(bsMat, mark_knots = "all")
title("B-spline basis functions")
```



As we described in the main manuscript, to enforce unimodality (i.e., the fact that the spline should have a single peak/ mode), we need to apply some restrictions on the spline coefficients $\alpha_1, \alpha_2, \alpha_3, \alpha_4$ and α_5 . We fix the general shape by requiring the coefficients to be non-decreasing up to the mode and then non-increasing (Köllmann, Bornkamp, and Ickstadt 2014). In practice, the α parameters are constrained to sum to one and we apply the constraint to the unconstrained coefficients β . For simplicity, we will focus on the α values here, just keep in mind that later the constraint will be applied to the unconstrained coefficients.

Because unimodality is enforced via constraints on the spline coefficients, and not directly on the function values or the location of the mode, we cannot predict in advance exactly where the mode of the spline function will be on the x-axis because the basis functions overlap. Thus a single coefficient doesn't control the function height at a single point on the x-axis, but rather influences the entire region where the associated basis function is non-zero. So even if we have an increasing then decreasing sequence of spline coefficients, the actual mode of the function might fall in the following locations: (1) In the first interval $[0.0, 0.5)$, (2) at the knot ($x = 0.5$), or (3) in the second interval $(0.5, 1.0)$, depending on the interaction of the coefficients.

Let us consider three sequences of spline coefficients, each with a mode at α_3 , but producing a mode in the function at different locations, with the function's mode location seen in the plot of the function, below.

```
# Define increasing then decreasing sequence of spline coefficients:
alpha1 <- c(2, 9, 10, 2, 0.1) # function's mode is in [0, 0.5)
spline1 <- bsMat %*% alpha1
alpha2 <- c(1, 2, 10, 2, 1)    # function's mode is at the knot, x = 0.5
spline2 <- bsMat %*% alpha2
alpha3 <- c(0.1, 2, 10, 9, 2)  # function's mode is in (0.5, 1]
spline3 <- bsMat %*% alpha3
```

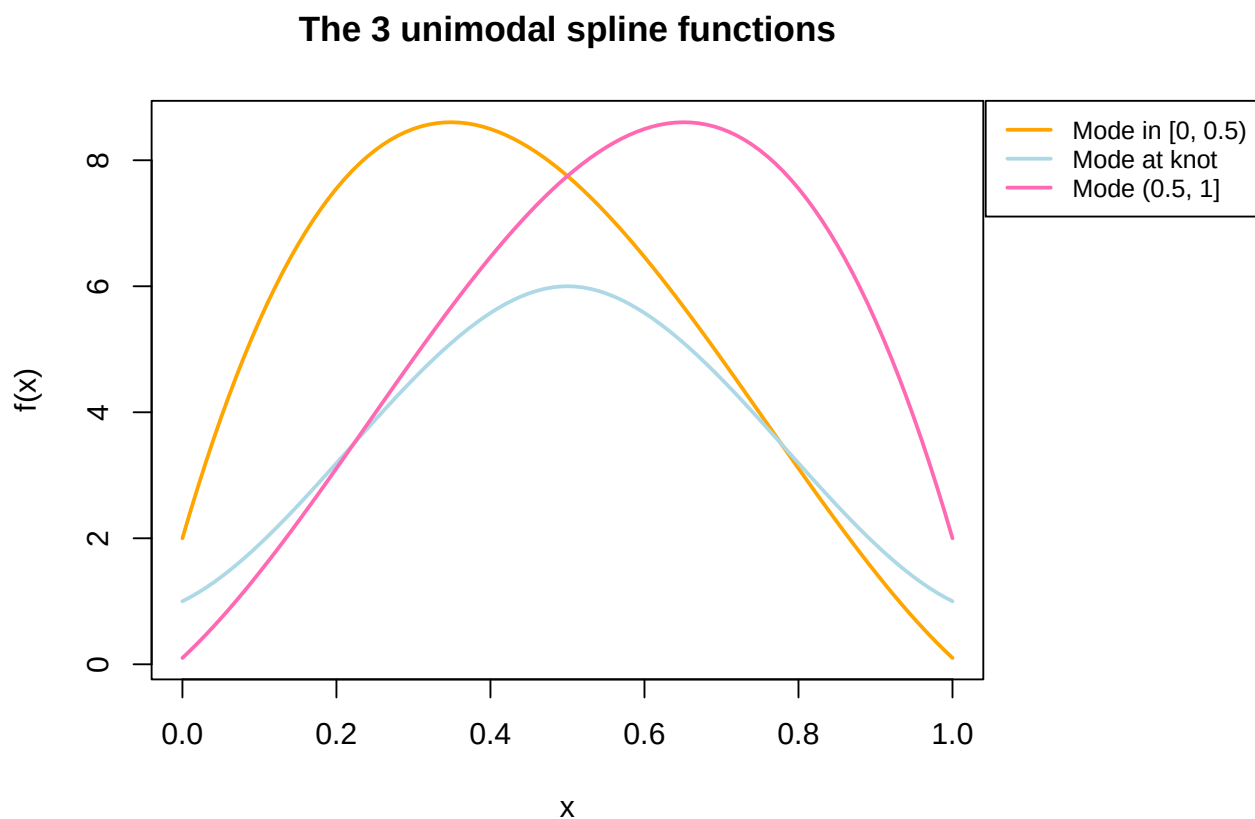
```

par(mar = c(5, 4, 4, 8), xpd = TRUE) # xpd=TRUE allows drawing outside plot

# Plot the functions
plot(x, spline1, type = "l", ylab = "f(x)", col = "orange", lwd = 2)
lines(x, spline2, col = "lightblue", lwd = 2)
lines(x, spline3, col = "hotpink", lwd = 2)
title("The 3 unimodal spline functions")

# Add the legend outside the plot
legend("topright", inset = c(-0.33, 0), # negative x inset pushes legend outside
      legend = c("Mode in [0, 0.5)", "Mode at knot", "Mode (0.5, 1]"),
      col = c("orange", "lightblue", "hotpink"),
      lwd = 2,
      cex = 0.8)

```



Observe that in all three cases, the peak value of the sequence $\alpha_1, \dots, \alpha_5$ occurs at the third coefficient. However, the mode of the function f (i.e., the location in the x -axis where the function reaches its maximum) differs across the three examples. This illustrates an important point: the location of the peak in f is influenced not only by where the α 's peak is in the sequence, but also by the shape and placement of the B-spline basis functions and the specific values of all coefficients. In other words, even with the same unimodality constraint on the coefficients, the interaction between the α 's and the basis functions can shift the location of the mode in the function output. This is something to keep in mind when interpreting or designing unimodal spline models.

3. Unimodal splines for HMMs

Now that we have a better intuition for the unimodality constraint and the mode's location, we will fit unimodal splines to a dataset. To demonstrate the application of nonparametric unimodal estimation for state-dependent densities within HMMs, we will use the `nessi` dataset which is part of the `LaMa` R package. For detailed instructions on how to use the `LaMa` package to fit nonparametric densities, refer to the `LaMa` vignette on Penalised Splines. We will work on the acceleration data of the Loch Ness Monster. More specifically, we will work on the overall dynamic body acceleration (ODBA) datastream. Here, we only use the first 1000 observations to speed up computations.

```
data <- nessi[1:1000, 1:2] # only using the first 1000 observations for this example
head(data)
```

```
##           ODBA    logODBA
## 1 0.03775025 -3.276763
## 2 0.05417830 -2.915475
## 3 0.03625247 -3.317248
## 4 0.01310802 -4.334531
## 5 0.05402441 -2.918319
## 6 0.06133794 -2.791357
```

The first column of the dataset contains said overall dynamic body acceleration (ODBA) of the monster as a measure of its activity level. Since ODBA is strictly positive with some extreme values, direct analysis can be difficult. Therefore, for our analysis, we consider the logarithm of ODBA as our observed process, which is precomputed in the second column of the dataset (`logODBA`). We will fit a three-state HMM with unimodal nonparametric state-dependent distributions.

First, let us specify the model we will fit to our data. To do so, we use the `smooth_dens_construct()` function, available in `LaMa`.

We set the following arguments:

- `"data"`: dataframe containing data streams for the HMM. These can be of different types (`real`, `positive`, `circular`). This information is then converted into a suitable B-spline basis for density estimation.
- `"par"`: nested named list of initial means and standard deviations/concentrations for each data stream. This information is converted into initial guesses for the spline coefficients to facilitate easier numerical estimation.
- `"type"`: For each data stream, either “real” for data on the reals, “positive” for data on the positive reals or “circular” for angular data. Needs to be a vector with length equal to the number of data streams in `data`.
- `"k"`: number of basis functions for each data stream
- `"knots"`: optional named list of knots vectors (including the boundary knots) to be used for basis construction. If not provided, the knots are placed equidistantly for “real” and “circular” and using polynomial spacing for “positive”.

`smooth_dens_construct()` then builds the P-spline design and penalty matrices for each data stream as well as a matrix of initial spline coefficients based on the provided parameters. The basis functions are standardised such that they integrate to one, which matches our constraints for density estimation.

```

# set the arguments
k <- 25 # basis dimension
par0 <- list(logODBA = list(mean = c(-4, -3, -2.5), # guesses for initial mean
                             sd = c(0.3, 0.5, 0.5))) # guesses for initial sd
# used to construct initial guesses of spline coefficients

modmat <- smooth_dens_construct(
  data["logODBA"], # only one data stream here
  par = par0, # initial guesses for mean and sd of logODBA (for each state)
  k = k # basis dimension
)

```

```
## logODBA
```

```
## Leaving out last column of the penalty matrix, fix the last spline coefficient at zero for identification
## Parameter matrix excludes the last column. Add a (fixed) zero column using 'cbind(coef, 0)' in your code

```

We can now have a look at what `smooth_dens_construct()` returns, contained in `modmat`: a nested list containing the design matrices `Z`, the penalty matrices `S`, the initial coefficients `coef`, the prediction design matrices `Z_predict`, the prediction grids `xseq`, and details for the basis expansion for each data stream. More details about `smooth_dens_construct()` are available in the function documentation.

```

# objects for model fitting
Z <- modmat$Z$logODBA # spline design matrix for logODBA
S <- modmat$S$logODBA # penalty matrix for logODBA
beta <- modmat$coef$logODBA # initial spline coefficients matrix of dimension N by k-1
w <- modmat$basis$logODBA$w # 1 / integral of the basis functions,
# needed for reweighting in the unimodality penalty (see main manuscript)

# objects for prediction / plotting
Z_p <- modmat$Z_predict$logODBA # prediction design matrix
xseq <- modmat$xseq$logODBA # prediction sequence of logODBA values
N <- 3 # number of states

```

Before minimising the negative penalised log likelihood, we need to define the initial parameter list `par0` and a `dat` list that contains the data and potential hyperparameters. The elements in the `dat` list are not estimated by the optimizer. Note that we specified the vector `m` containing the indices where the signs of the spline coefficients change, for each state. This choice was based on preliminary analysis. In general, the mode positions should always be explored, which we will address in the next section.

```

# coefficient mode position for each state
m <- c(8, 12, 15) # one index per state, index <= 25, the number of alphas used

# initialise parameters
par0 <- list(beta0 = rep(-2, 6), # initial transition matrix on logit scale
             beta = beta) # spline coefficients prepared by buildSmoothDens()

# initialise data and hyperparameters
mydat <- list(N = N, # number of states
              Z = Z, # spline design matrix
              S = S, # spline penalty matrix
              lambda = rep(10, 3), # initial smoothness penalty parameters
              m = m, # coefficient mode indices
)

```

```
kappa = 1e4, # strength parameter of the unimodality penalty
logweights = t(log(w[-k] / w[k]))[rep(1, N), ] # weights
```

Observe that `beta` has 24 columns, instead of 25. This is because the last column, which corresponds to the last coefficient for each state, needs to be fixed to zero for identifiability which we do in the seventh line of `pnll` below using `cbind(beta, 0)`. Then, we transform the unconstrained parameter matrix to non-negative weights that sum to one – getting α as in the main manuscript – for each state using the inverse multinomial logistic link. The columns of the `allprobs` matrix are then computed as linear combinations of the columns of `Z` and the weights `alpha`. We then penalise the unconstrained coefficients `beta` using the `penalty()` function. This helps prevent the estimated densities from overfitting by becoming too wiggly. Finally, we apply the unimodality constraint on the `beta`'s using the `penalty_uni()` function. Note that we place the unimodality penalty on the reweighted coefficients `beta + logweights`. This is because the penalty needs to be on the scale of the original B-spline basis, which was standardised to integrate to one by `smooth_dens_construct()`. The reweighting is done to ensure that the unimodality constraint is applied correctly, as described in the main manuscript (Equation (5)).

```
pnll <- function(par){
  getAll(par, dat)
  ## state process model
  Gamma <- tpm(beta0)
  delta <- stationary(Gamma)
  ## state-dependent process model
  # multinomial logit for density weights
  alpha <- exp(cbind(beta, 0)) # last column is fixed at zero
  alpha <- alpha / rowSums(alpha); REPORT(alpha) # reporting for easy access
  # state-dependent densities
  allprobs <- matrix(1, nrow(Z), N)
  ind <- which(!is.na(Z[,1])) # NA handling
  # now just linear combinations of precomputed basis functions in Z
  allprobs[ind,] <- Z[ind,] %*% t(alpha) # this calculates Z %*% alpha[i,] for all states at once
  # forward algorithm + smoothness penalty + unimodality penalty
  - forward(delta, Gamma, allprobs) +
    penalty(beta, S, lambda) +
    penalty_uni(beta + logweights, m, kappa) # reweighting based on eq (5) in the main manuscript
}
```

We jointly estimate the model coefficients and the optimal degree of smoothness using the `qrem1()` function, an approximate restricted maximum likelihood procedure (Koslik 2024). See the LaMa vignette on Penalised Splines for more details.

```
mod <- qrem1(pnll, par0, mydat,
  random = "beta") # telling the model what are spline coefficients/ random effects
```

```
## Creating AD function
## Initialising with lambda: 10 10 10
## outer 1 - lambda: 5.037 4.858 7.431
## outer 2 - lambda: 2.731 3.536 6.597
## outer 3 - lambda: 1.641 3.509 6.33
## outer 4 - lambda: 1.15 3.545 6.246
## outer 5 - lambda: 0.917 3.579 6.22
## outer 6 - lambda: 0.801 3.602 6.212
## outer 7 - lambda: 0.742 3.616 6.21
```

```
## outer 8 - lambda: 0.71 3.625 6.21
## outer 9 - lambda: 0.694 3.63 6.21
## outer 10 - lambda: 0.685 3.633 6.21
## outer 11 - lambda: 0.68 3.634 6.21
## outer 12 - lambda: 0.677 3.635 6.21
## outer 13 - lambda: 0.676 3.635 6.21
## outer 14 - lambda: 0.675 3.636 6.21
## outer 15 - lambda: 0.675 3.636 6.21
## Converged
## Final model fit with lambda: 0.675 3.636 6.21
```

Now let us have a look at the model output, using the fitted results contained in `mod`.

```
# Get the estimated (constrained) spline coefficients
Alphas <- mod$alpha

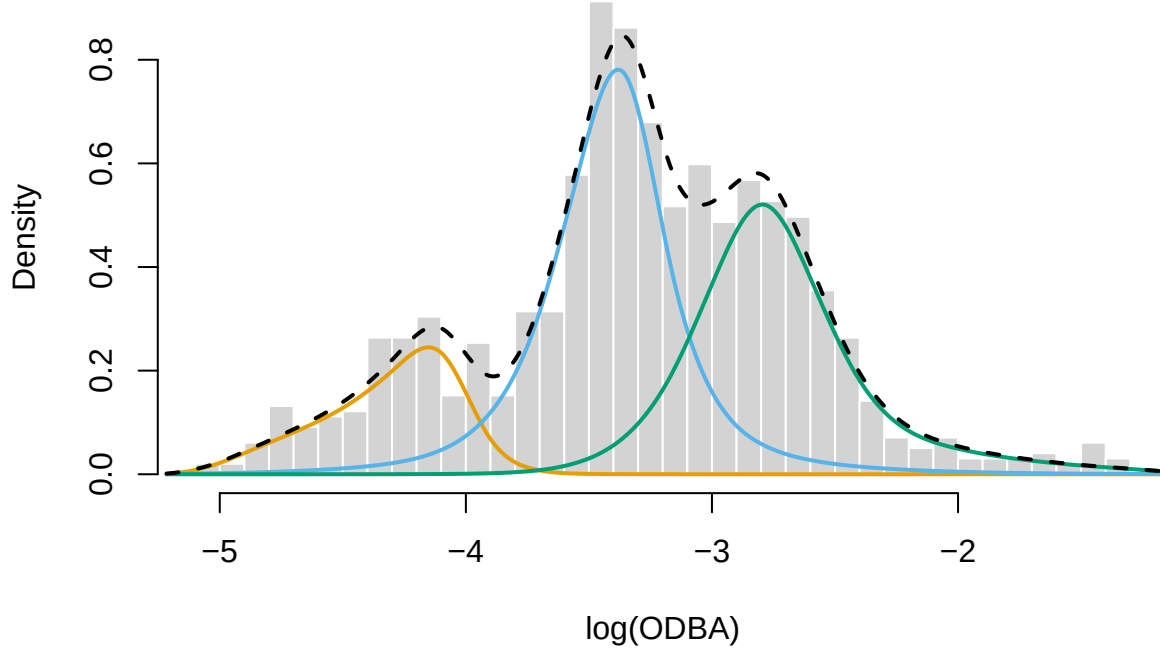
# Compute the state-dependent densities on prediction grid
Dens <- Z_p %*% t(Alphas)

# Get the stationary distribution
Deltas <- mod$delta

# Compute marginal density
marDens <- colSums(t(Dens) * Deltas)

# Plot histograms of data and density estimates
color <- c("#E69F00", "#56B4E9", "#009E73")
par(mfrow = c(1,1))
hist(data$logODBA, prob = TRUE, breaks = 50, bor = "white",
      main = "Estimated state-dependent densities", xlab = "log(ODBA)")
for(i in 1:N) lines(xseq, Deltas[i] * Dens[,i], col = color[i], lwd = 2)
lines(xseq, marDens, lty = 2, lwd = 2)
```

Estimated state-dependent densities



The unimodality constraint is implemented in the last line of `pnll` by calling the function `penalty_uni()`. Internally, this function applies the following strategy, described in the manuscript. The linear positivity constraint is written as

$$C'\beta \geq 0.$$

Instead of enforcing this as a hard constraint, we incorporate it as a penalty in the objective function by adding

$$\min(C'\beta, 0),$$

scaled by a large penalty parameter (called `kappa` in `penalty_uni()`) to strongly discourage violations. Since the standard minimum function is not differentiable at 0, we use a smooth approximation, specifically

$$\min(x, 0) \approx -\frac{1}{\rho} \log[1 + \exp(-\rho x)],$$

implemented in the function `min0_smooth()`, to ensure continuity and differentiability in the optimization process. The parameter `rho` (defaulting to 20) controls the smoothness of the approximation: the larger, the closer to the minimum function, as we can see in the plot below.

```
## Plotting the smooth approximation for different rho values

# Sequence of x values
x_vals <- seq(-1, 1, length.out = 400)

# List of rho values to try
rhos <- c(1, 5, 10, 20, 50, 100)

# Set up the plot
plot(x_vals, min0_smooth(x_vals, rho = rhos[1]), type = "l",
```



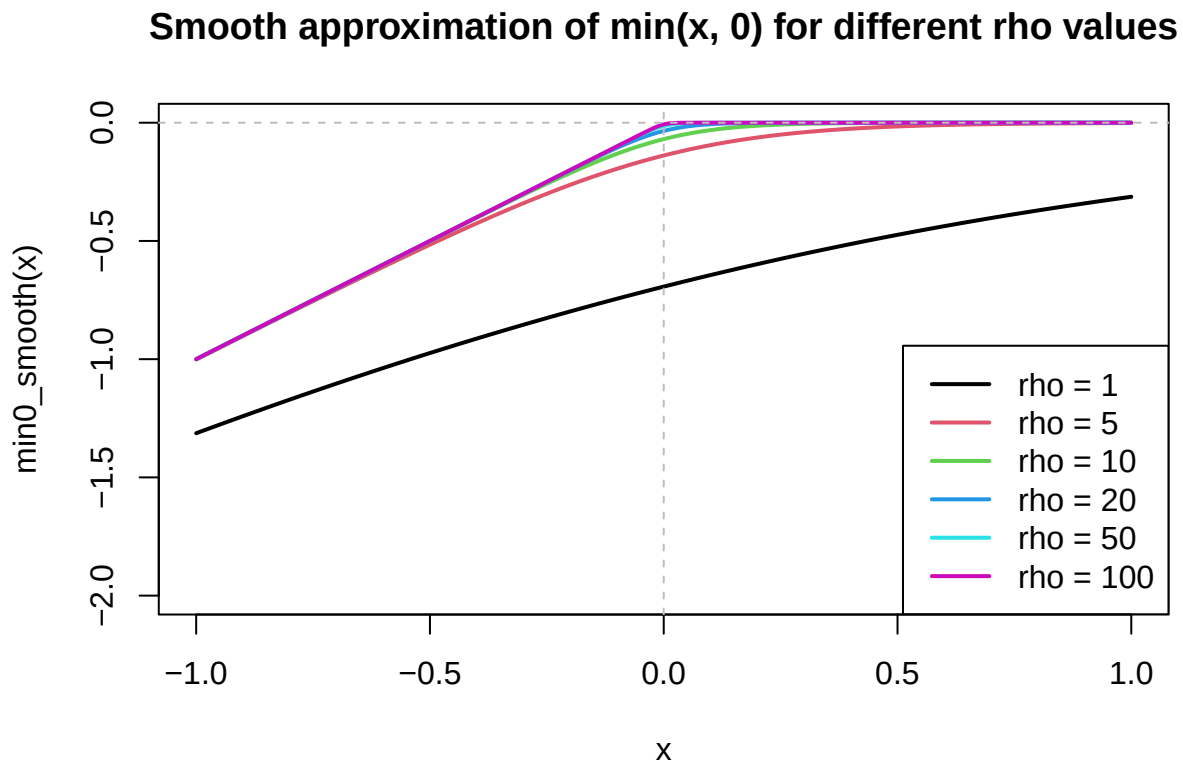
```

ylim = c(-2, 0), lwd = 2, col = 1,
xlab = "x", ylab = "min0_smooth(x)",
main = "Smooth approximation of min(x, 0) for different rho values")

# Add curves for other alpha values
for (i in 2:length(rhos)) {
  lines(x_vals, min0_smooth(x_vals, rho = rhos[i]), col = i, lwd = 2)
}

# Add legend
legend("bottomright", legend = paste("rho =", rhos), col = 1:length(rhos), lwd = 2)
abline(h = 0, v = 0, col = "gray", lty = 2)

```



Now we are ready to fit the whole model and search for the mode instead of specifying it subjectively!

4. Search for the mode

Using the fit with the three selected modes contained in `m`, we can identify potential indices for the change of sign in the unimodality constraint, which allows us to avoid exploring the entire range of possible modes. We assign the states in increasing order: the first state corresponds to the leftmost mode, the second to the central mode, and the third to the rightmost mode.

```

# pick candidates manually
m1 <- 5:8   # Potential constraints indices for the first state
m2 <- 10:13 # Potential constraints indices for the second state
m3 <- 14:16 # Potential constraints indices for the third state

```

Then we can build the grid of exploration and set up the model as before.

```

m_grid <- expand.grid(m1 = m1, m2 = m2, m3 = m3)

mydat <- list(N = N, # number of states
             Z = Z, # spline design matrix
             S = S, # spline penalty matrix
             lambda = rep(10, 3), # initial smoothness penalty parameters
             kappa = 1e4,
             logweights = t(log(w[-k] / w[k]))[rep(1, N), ]) # weights

```

We can then fit a model for each combination of indices. In total, this involves fitting $\text{length}(m1) \times \text{length}(m2) \times \text{length}(m3)$ models, which amounts to 48 in our case. For convenience, we currently explore only one randomly selected initial value `par0`, but ideally, we should examine more, meaning even more models to fit.

```

# maximizing the likelihood for each set of mode locations
llks <- rep(NA, nrow(m_grid))
for(i in 1:nrow(m_grid)){
  mydat$m <- m_grid[i, ]
  thismod <- qrem1(pnll, par0, mydat, random = "beta", silent = 2)
  llks[i] <- thismod$llk
}

```

```

# Find the mode estimates with largest likelihood
(m_opt <- m_grid[which.max(llks), ])

```

```

##      m1 m2 m3
## 27    7 12 15

```

```

# Update the optimal mode
mydat$m <- m_opt

# Fit the model
mod_c <- qrem1(pnll, par0, mydat, random = "beta")

```

```

## Creating AD function
## Initialising with lambda: 10 10 10
## outer 1 - lambda: 5.48 5.082 7.432
## outer 2 - lambda: 3.489 3.541 6.595
## outer 3 - lambda: 2.48 3.484 6.324
## outer 4 - lambda: 1.92 3.501 6.238
## outer 5 - lambda: 1.61 3.525 6.211
## outer 6 - lambda: 1.443 3.543 6.204
## outer 7 - lambda: 1.358 3.554 6.201
## outer 8 - lambda: 1.305 3.562 6.202
## outer 9 - lambda: 1.278 3.566 6.202
## outer 10 - lambda: 1.262 3.568 6.201
## outer 11 - lambda: 1.254 3.569 6.201
## outer 12 - lambda: 1.251 3.57 6.201
## outer 13 - lambda: 1.249 3.57 6.201
## outer 14 - lambda: 1.247 3.571 6.201
## outer 15 - lambda: 1.246 3.571 6.201
## outer 16 - lambda: 1.246 3.571 6.201

```

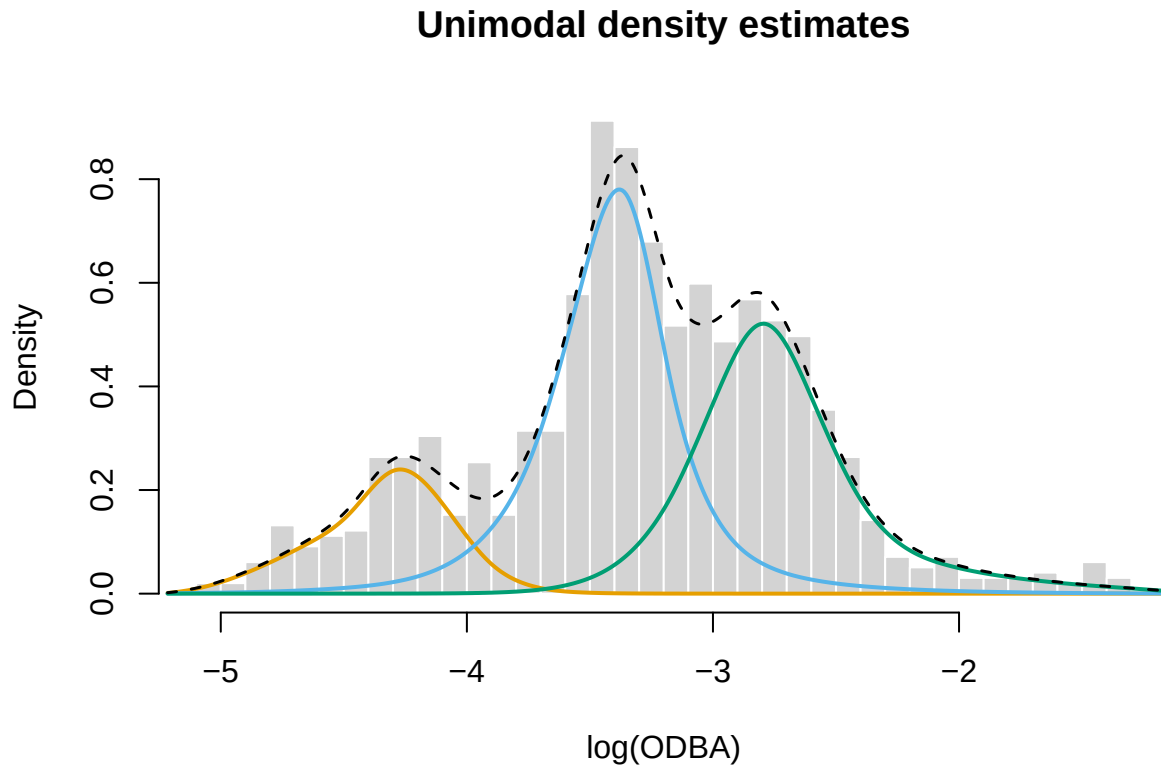
```
## Converged
## Final model fit with lambda: 1.246 3.571 6.201
```

```
# Calculate the unimodal densities
alpha_c <- mod_c$alpha
delta_c <- mod_c$delta
dens_c <- t(delta_c * t(Z_p %*% t(alpha_c)))
```

Now, we can have a look at our final estimated state-dependent densities.

```
# plot state-dependent and marginal distributions
par(mfrow = c(1,1))

hist(data$logODBA, breaks = 30, prob = TRUE, bor = "white",
     main = "Unimodal density estimates",
     xlab = "log(ODBA)", ylab = "Density")
for(i in 1:3) lines(xseq, dens_c[,i], col = color[i], lwd = 2)
lines(xseq, rowSums(dens_c), lty = 2, lwd = 1.5)
```



References

- Köllmann, Claudia, Björn Bornkamp, and Katja Ickstadt. 2014. “Unimodal Regression Using Bernstein–Schoenberg Splines and Penalties.” *Biometrics* 70 (4): 783–93.
- Koslik, Jan-Ole. 2024. “Efficient Smoothness Selection for Nonparametric Markov-Switching Models via Quasi Restricted Maximum Likelihood.” *arXiv Preprint arXiv:2411.11498*.